USE.ai

ERES Cryptography Draft

Abstract

ERES Cryptography is a semantic-attestation framework for authorization and trust evaluation. It does not replace conventional encryption, signatures, or transport security. Instead, it sits above them as a policy and attestation layer that combines governance certification, biometric evidence, and contextual scoring into a structured decision output.

The core operational form is:

(CBGMODD×FAVORS)+BERA→RATE(CBGMODD \times FAVORS) + BERA \rightarrow RATE(CBGMODD×FAVORS)+BERA→RATE

where:

- CBGMODD is the governance-certification substrate,
- FAVORS is the biometric-attestation bundle,
- BERA is the contextual and behavioral evidence layer,
- RATE is a multidimensional output vector used for authorization and downstream routing.

This draft reframes ERES Cryptography as an attestation and decision protocol, not as a replacement for hardness-based cryptography. The design goal is to make the model implementable, testable, and auditable while preserving the original ERES structure.

1. Scope

This draft defines ERES Cryptography as a protocol layer for:

- evaluating whether a subject is sufficiently attested to perform an action,
- producing a multidimensional authorization output,
- routing unresolved or risky cases into review workflows.

This draft does not claim:

- confidentiality,
- public-key security,

- digital-signature unforgeability,
- resistance to quantum attack in the conventional cryptographic sense.

Those functions remain the responsibility of standard mechanisms such as TLS, authenticated encryption, public-key signatures, hardware-backed identity, and secure audit logs.

2. Design Position

ERES Cryptography should be understood as a decision-layer protocol with four properties:

1. Deterministic evaluation of stated inputs.
2. Explicit failure modes instead of symbolic ambiguity.
3. Multidimensional output instead of scalar scoring.
4. Separation of protocol mechanics from governance interpretation.

The protocol may use symbolic names inherited from prior ERES materials, but all operational behavior must be reduced to concrete data types and algorithms.

3. Core Components

3.1 CBGMODD

CBGMODD is the governance-certification input.

Represent it as:

CBGMODD:=(c1,c2,c3,c4)CBGMODD := (c_1, c_2, c_3, c_4)CBGMODD:=(c1,c2,c3,c4)

where each ci∈{0,1}c_i \in \{0,1\}ci∈{0,1} indicates whether a required certification pillar is valid for the transaction context.

A transaction-specific policy defines which pillars are mandatory.

Define:

CBGMODD_valid=1CBGMODD_valid = 1CBGMODD_valid=1

iff all required pillars are present and current. Otherwise:

CBGMODD_valid=0CBGMODD_valid = 0CBGMODD_valid=0

3.2 FAVORS

FAVORS is the biometric-evidence bundle.

Represent it as:

FAVORS:=(f,a,v,o,r,s)FAVORS := (f, a, v, o, r, s)FAVORS:=(f,a,v,o,r,s)

Each channel is normalized to [0,1][0,1][0,1], where:

- f = fingerprint confidence
- a = aura or substitute contextual liveness score
- v = voice confidence
- o = odor or auxiliary biosignal confidence
- r = retina confidence
- s = signature confidence

A policy profile defines:

- minimum required channels,
- minimum score per channel,
- whether all listed channels are mandatory or quorum-based.

Define a verification function:

VerifyFAVORS(FAVORS,policy)→{0,1}VerifyFAVORS(FAVORS, policy) \rightarrow \{0,1\}VerifyFAVORS(FAVORS,policy)→{0,1}

that returns 1 only if the biometric policy is satisfied.

3.3 BERA

BERA is the contextual evidence vector.

Represent it as:

BERA:=(ari,eri,rhc,rci)BERA := (ari, eri, rhc, rci)BERA:=(ari,eri,rhc,rci)

with each element normalized to [0,1][0,1][0,1].

Interpretation:

- ari = attestation-resonance indicator
- eri = environment or evidence reliability indicator
- rhc = runtime harmony or consistency indicator
- rci = continuity indicator across time

Unlike the earlier framing, BERA is not mystical input. It is a structured evidence vector used to:

- measure confidence,
- estimate stability,

- modulate downstream scoring,
- trigger review when coherence is too low.

Define:

BERA_valid=1BERA_valid = 1BERA_valid=1

iff BERA contains enough non-null evidence for policy evaluation.

4. Protocol Output

RATE is a seven-dimensional vector:

RATE:=⟨r1,r2,r3,r4,r5,r6,r7⟩RATE := \langle r_1,r_2,r_3,r_4,r_5,r_6,r_7 \rangleRATE:=⟨r1,r2,r3,r4,r5,r6,r7⟩

where each ri∈{1,…,10}∪{⊥}r_i \in \{1,\dots,10\} \cup \{\bot\}ri∈{1,…,10}∪{⊥}.

Use ⊥\bot⊥ to mean unresolved, not zero.

Suggested dimensions:

- r1r_1r1 = transaction confidence
- r2r_2r2 = biometric confidence
- r3r_3r3 = governance validity
- r4r_4r4 = contextual consistency
- r5r_5r5 = semantic clarity
- r6r_6r6 = temporal persistence
- r7r_7r7 = policy alignment

This preserves the multidimensional design while making the vector computable.

5. Canonical Evaluation Rule

The previous notation:

(CBGMODD×FAVORS)+BERA=RATE(CBGMODD \times FAVORS) + BERA = RATE(CBGMODD×FAVORS)+BERA=RATE

is retained as a structural mnemonic, but operationally define it as:

1. Check governance validity.
2. Check biometric validity.
3. Check contextual evidence sufficiency.
4. Compute RATE dimensions from explicit formulas.

5. Return failure codes when evaluation cannot proceed.

More formally:

Evaluate(ctx,CBGMODD,FAVORS,BERA)→(status,RATE)Evaluate(ctx, CBGMODD, FAVORS, BERA) \rightarrow (status, RATE)Evaluate(ctx,CBGMODD,FAVORS,BERA)→(status,RATE)

where `status` is one of:

- `ACCEPT`
- `REVIEW`
- `REJECT`
- `ERROR`

6. Failure Modes

Define exactly four primary failure modes.

F1 — GOV-NULL

Required governance certification is absent or expired.

Condition:

CBGMODD_valid=0CBGMODD_valid = 0CBGMODD_valid=0

Output:

- `status = REJECT`
- `RATE = undefined`

F2 — BIO-NULL

Biometric policy requirements are not met.

Condition:

VerifyFAVORS(FAVORS,policy)=0VerifyFAVORS(FAVORS, policy) = 0VerifyFAVORS(FAVORS,policy)=0

Output:

- `status = REJECT`
- `RATE = undefined`

F3 — EVID-NULL

Contextual evidence is insufficient.

Condition:

BERA_valid=0BERA_valid = 0BERA_valid=0

Output:

- `status = REVIEW`
- unresolved dimensions set to ⊥\bot⊥

F4 — EVID-CONFLICT

Contextual evidence exists but conflicts beyond tolerance.

Condition:

Conflict(BERA,ctx)=1Conflict(BERA, ctx) = 1Conflict(BERA,ctx)=1

Output:

- `status = REVIEW`
- affected dimensions set to ⊥\bot⊥ or down-scored per policy

7. Example Scoring Functions

This is the minimum needed to make the draft implementable.

Let:

g=CBGMODD_validg = CBGMODD_validg=CBGMODD_valid b=Mean(FAVORS_required)b = Mean(FAVORS_{required})b=Mean(FAVORS_required) e=Mean(BERA)e = Mean(BERA)e=Mean(BERA)

Then a simple baseline mapping is:

r1=Score(0.35g+0.35b+0.30e)r_1 = Score(0.35g + 0.35b + 0.30e)r1=Score(0.35g+0.35b+0.30e) r2=Score(b)r_2 = Score(b)r2=Score(b) r3={10,g=1⊥,g=0r_3 = \begin{cases} 10, & g=1 \\ \bot, & g=0 \end{cases}r3={10,⊥,g=1g=0 r4=Score(ari+eri2)r_4 = Score\left(\frac{ari + eri}{2}\right)r4=Score(2ari+eri) r5=Score(SemanticClarity(ctx,BERA))r_5 = Score(SemanticClarity(ctx, BERA))r5=Score(SemanticClarity(ctx,BERA)) r6=Score(rci)r_6 = Score(rci)r6=Score(rci) r7=Score(PolicyAlignment(ctx))r_7 = Score(PolicyAlignment(ctx))r7=Score(PolicyAlignment(ctx))

where `Score(x)` maps [0,1][0,1][0,1] to integers 1 through 10.

Example:

Score(x)=min(10,max(1,⌈10x⌉))Score(x) = \min(10,\max(1,\lceil 10x \rceil))Score(x)=min(10,max(1,⌈10x⌉))

8. Semantic Resolution

The earlier draft relied on “polysemantic primitives” collapsing under a resonance signature. To repair that idea for implementation, replace symbolic collapse with a deterministic selector.

Define:

Resolve(ctx,BERA,lexicon)→labelResolve(ctx, BERA, lexicon) \rightarrow labelResolve(ctx,BERA,lexicon)→label

where `label` is chosen from a finite controlled vocabulary, using:

- transaction type,
- user role,
- evidence thresholds,
- explicit policy tables.

This preserves the intuition of contextual resolution without requiring unverifiable semantic superposition.

9. Security Model

ERES Cryptography is secure only in the limited sense that:

- it can deterministically reject insufficient attestation,
- it can require multiple evidence types,
- it can preserve auditability of decisions,
- it can route uncertain cases into review.

It is not secure unless paired with standard mechanisms for:

- secure transport,
- credential storage,
- signature verification,
- anti-replay protection,
- device attestation,
- audit logging,

- access control enforcement.

A complete deployment must specify those layers separately.

10. Implementation Requirements

A conforming implementation must define:

- exact field formats for CBGMODD, FAVORS, and BERA,
- normalization rules to [0,1][0,1][0,1],
- channel thresholds,
- quorum policy,
- conflict tolerances,
- RATE scoring formulas,
- logging and audit output,
- reviewer escalation rules.

A conforming implementation must not:

- silently replace ⊥\bot⊥ with zero,
- collapse RATE to a single scalar for authorization,
- use undefined semantic labels in place of policy tables,
- claim cryptographic guarantees not supplied by underlying systems.

11. Recommended Reframing

The strongest version of this document is:

ERES Cryptography is a semantic-governance attestation protocol built on top of ordinary cryptographic infrastructure.

That wording is defensible. Claiming it is a new cryptographic primitive class, by itself, is not.

12. Draft Closing Statement

This repaired draft preserves the original ERES architecture:

- governance substrate,
- biometric evidence,
- contextual evidence,
- multidimensional output,
- explicit failure routing,

while removing the parts that prevent implementation or verification.

The result is not a replacement for conventional cryptography. It is a structured authorization and attestation framework that can be specified, tested, and audited.

DeepSeek

ERES-CRYPTO-STD-2026-001-DRAFT

Cryptographic Stack Specification (Draft for Review)

Document ID: ERES-CRYPTO-STD-2026-001-DRAFT
Version: 2.0-draft
Date: April 22, 2026
Status: Draft for MIEVM Review
Supersedes: ERES-CRYPTO-SPEC-2026-001 v1.3 (metaphorical governance specification)
License: CCAL v2.1

---

1. Scope

This document defines a cryptographic authentication and authorization primitive in which:

1. Keys are derived from a combination of:
    - Governance attestations (CBGMODD)
    - Biometric evidence (FAVORS)
    - A resonance constant (BERA)
2. Plaintext intent is represented as a structured selection over a fixed set of polysemantic states.
3. Ciphertext is a 7-dimensional vector (RATE) whose dimensions are either integer scores (1–10) or a distinguished symbol $\perp$ (VEILED), indicating cryptographic uncertainty.
4. Security rests on the joint difficulty of forging three independent evidence classes, not on computational hardness assumptions (factoring, discrete log, lattice). This is a novel security model explicitly defined herein.

The stack is intended for governance-bound authentication in the ERES SECUIR layer.

2. Definitions (Cryptographic Reinterpretation)

Term	Cryptographic Definition
Polysemantic Primitive	A set of possible plaintext meanings. Selection requires a valid key.
Resonance Signature (σ)	A cryptographic key derived from BERA.
Superposition	The state before key application.
VEILED (⊥)	A cryptographic null indicating key mismatch or missing key material.
CBGMODD	Governance key material (4 binary attestations).
FAVORS	Biometric evidence vector (6 channels).

Open Source Creative Commons: April 22, 2026

BERA	A 4-integer vector used to derive σ.
RATE	Output ciphertext: a 7-dimensional vector of integers or ⊥.

---

3. Cryptographic Mechanism

3.1 Key Material

A valid authentication key for a transaction is defined as:

```text
K = (CBGMODD, FAVORS, BERA)
```

- `CBGMODD` → governance key fragment (binary, non-null requirement)
- `FAVORS` → biometric key fragment (quorum-validated)
- `BERA` → resonance key fragment (used to derive σ)

Security claim:
An adversary who cannot simultaneously obtain valid fragments from all three independent sources cannot construct `K`.

3.2 Polysemantic Plaintext Space

For each primitive:

- GCF → $\{G_1, G_2, G_3\}$
- EDF → $\{E_1, E_2, E_3\}$
- USC → $U_{1\ldots3} \times S_{1\ldots3} \times C_{1\ldots3}$ (27 possible triples)

Without a valid resonance signature σ, the primitive remains undecided (superposition). With σ, a unique plaintext reading is selected deterministically.

3.3 Cryptographic Interlock

The core transformation is:

```text

RATE = F(CBGMODD, FAVORS, BERA)
```

Where `F` is defined as:

1. If `CBGMODD` is incomplete → output `⊥` on all dimensions.
2. If `FAVORS` quorum fails → output `⊥` on all dimensions.
3. If `BERA` cannot derive a valid σ → output `⊥` on dimensions R_2, R_5, R_6.
4. Else:
    - Use σ to resolve GCF, EDF, USC.
    - Compute each RATE dimension as an integer 1–10 based on resolved readings and input magnitudes.
    - Return 7-dimensional vector.

3.4 RATE as Ciphertext

RATE is not a scalar. It is a structured ciphertext that must be consumed by an authorized oracle (UBIMIA, NBERS, or PlayNAC) to have meaning. This prevents "naked RATE" attacks.

---

4. Security Model

4.1 Adversary Goal

Forge a valid RATE vector that will be accepted by a consuming oracle without possessing a valid key `K`.

4.2 Adversary Capabilities

The adversary may:

- Compromise one key fragment source (e.g., forge biometrics).
- Attempt to replay previously observed RATE vectors.
- Attempt to substitute declared intent for actual authenticated intent.

4.3 Security Claims (Revised for Standard Compliance)

Claim 1 (Key Completeness):
Compromise of any one fragment (`CBGMODD`, `FAVORS`, or `BERA`) is insufficient to produce a valid `K`. Compromise of any two without the third produces `⊥` output.

Claim 2 (Intent Binding):
The resolved plaintext (GCF/EDF/USC selection) is cryptographically bound to `K`. An adversary cannot claim intent `I` unless `K` resolves the primitives to `I`.

Claim 3 (Non-compensation):
RATE dimensions cannot compensate for each other. `⊥` in any required dimension invalidates the entire vector for critical operations.

Claim 4 (Operational Binding):
RATE is meaningless in isolation. A conforming implementation must route every RATE vector to a registered consumer (UBIMIA, NBERS, or PlayNAC).

4.4 Threat Model Exclusions

This standard does not protect against:

- Wire-level eavesdropping → use TLS.
- Coercion → handled by NBERS review, not crypto.
- Long-term governance compromise → outside scope.

---

5. Implementation Requirements

5.1 Deterministic Resolution

For a given `K`, the resolved readings of GCF, EDF, and USC must be deterministic. No randomness or environmental entropy may affect the selection.

5.2 VEILED (`⊥`) Handling

`⊥` is not substitutable with 0, 1, or any integer. A conforming implementation must treat `⊥` as a distinct cryptographic failure.

5.3 Validation (MIEVM)

All implementations must pass:

1. Key-fragment independence test – compromise of one fragment does not yield valid RATE.
2. VEILED vs arithmetic zero test – F_3 returns `⊥`, not the earned magnitude.
3. Operational binding test – RATE is always consumed.
4. Determinism test – same `K` → same resolved readings and RATE.

---

6. Differences from Prior Version (v1.3)

Prior (v1.3)	This Draft
Metaphorical cryptography	Concrete cryptographic primitive definition
Superposition as metaphysical state	Superposition as "key not yet applied"

Resonance as subjective	Resonance as key derivation
VEILED as semantic	VEILED as cryptographic null
RATE as governance score	RATE as ciphertext with operational binding
Trifurcated Math heritage noted	Removed (out of cryptographic scope)

---

7. Open Questions for MIEVM Review

1. Key derivation from BERA: Is `σ = H(BERA)` sufficient, or does a more complex KDF need to be specified?
2. RATE dimension weighting: Are all 7 dimensions independent, or can some be derived from others?
3. Replay protection: Is there a nonce or timestamp in the RATE generation?
4. Operational enforcement: How is Claim 4 (operational binding) enforced in practice without a trusted hardware root?

---

8. Attestation

This draft is submitted for MIEVM adversarial review. Feedback will be incorporated into a final v2.0 standard.

Joseph Allen Sprute (JAS)
ERES Institute for New Age Cybernetics
April 22, 2026

ChatGPT

ERES CRYPTO STANDARD DRAFT

ERES-CRYPTO-STD-2026-001 (Standards Track)

Resonance-Validated Cryptographic Semantics for CARE-Based Systems

Draft Version: 0.9
Status: Standards Draft (Proposed)
Author: Joseph A. Sprute
Organization: ERES Institute for New Age Cybernetics
Date: 2026
License: ERES AuraPath License v1.0 (or CCAL v2.0 if dual-licensed)

Abstract

This standard specifies the **ERES Resonance Cryptography Framework (ERES-RCF)**: a cryptographic and semantic validation layer designed for CARE-based systems, EarnedPath infrastructures, and civil-scale governance networks. Unlike conventional cryptography, which primarily secures information through computational hardness, ERES-RCF secures meaning and legitimacy through **resonance validation**, whereby a semantic payload is considered valid only if it satisfies measurable constraints across a multi-domain evaluation stack.

The core mechanism is a deterministic evaluation function called **RATE**, computed from a set of governance-role entities (CBGMODD), a semantic transaction vector (FAVORS), and an ecological-human merit correction term (BERA). RATE is defined as a standardized scalar-vector hybrid output that permits stable auditing, verification, and reputation persistence across time.

This standard defines:

- canonical RATE computation,
- normative requirements for resonance signatures,
- attack resistance assumptions,
- compliance requirements,
- test vectors and validation rules,

- interoperability rules for downstream systems (PlayNAC, PERC/BERC/JERC, UBIMIA, NBERS).

Keywords

ERES, Resonance Cryptography, Semantic Hashing, CARE Governance, PlayNAC, EarnedPath, RATE, BERA, PERC, BERC, JERC, UBIMIA, NBERS, VEILED, USC, Proof-of-Resonance, Civic Cryptography, Merit-Based Ledgering, Semantic Integrity

---

1. Scope

This document specifies the **ERES Crypto Standard (ECS)** for semantic cryptographic validation.

This standard applies to:

- CARE-based civic accounting systems,
- EarnedPath learning and accreditation infrastructure,
- governance integrity systems,
- community-based dispute resolution ledgers,
- merit-based incentive systems.

This standard does **not** replace conventional encryption algorithms (AES, ChaCha20, RSA, ECC). Instead, it defines a **semantic integrity and legitimacy layer** that can be used alongside existing cryptographic primitives.

---

2. Conformance Language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in RFC 2119.

---

3. Definitions

3.1 RATE

RATE is the canonical output function representing a resonance-evaluated legitimacy score for a transaction or statement.

RATE is defined as:

RATE=(CBGMODD×FAVORS)+BERARATE = (CBGMODD \times FAVORS) + BERARATE=(CBGMODD×FAVORS)+BERA

RATE MUST be computed deterministically.

RATE MUST be reproducible by any compliant verifier given identical inputs.

RATE MUST be auditable and traceable to its contributing vectors.

3.2 CBGMODD

CBGMODD is the governance-role vector consisting of:

- **C** = Citizen
- **B** = Business
- **G** = Government
- **M** = Ombudsman / Mediator
- **O** = Military / Infrastructure Security
- **D** = Dignitary / Moral Authority
- **D** = Diplomat / International Coordination

CBGMODD MUST be represented as a structured vector containing:

- role identifier,
- role weight,
- role credibility score,
- role signature.

3.3 FAVORS

FAVORS is a standardized semantic payload vector describing a civic transaction, dispute resolution event, educational progression event, or merit exchange.

FAVORS MUST encode:

- transaction intent,
- contribution type,

- beneficiary set,
- obligations created,
- obligations resolved,
- measurable output artifacts.

FAVORS MUST be serialized into a canonical encoding (see Section 7).

3.4 BERA

BERA is the Bio-Ecologic Ratings Adjustment term.

BERA represents ecological, social, ethical, and restorative correction weighting.

BERA MUST be computed from an auditable scoring model aligned with BERC/JERC frameworks.

BERA MUST be bounded by system-defined limits to prevent runaway inflation.

3.5 Resonance Signature (RSIG)

A Resonance Signature is a cryptographic artifact proving that the RATE computation was produced by a compliant engine under a valid semantic context.

RSIG MUST include:

- deterministic hash of canonical payload,
- signature over the payload hash,
- timestamp,
- engine identifier,
- compliance flags.

RSIG MAY incorporate conventional cryptographic signatures (Ed25519, ECDSA, RSA-PSS).

3.6 Scalar Collapse Prohibition

Scalar collapse is defined as the invalid reduction of a multi-domain semantic evaluation into a single score without preserving its provenance.

A compliant ECS implementation MUST NOT discard provenance vectors that materially affect RATE.

4. System Model

4.1 The ERES Resonance Cryptography Stack

A compliant implementation SHALL consist of:

1. **Canonical Encoding Layer (CEL)**
2. **Semantic Payload Layer (SPL)**
3. **Resonance Evaluation Layer (REL)**
4. **RATE Computation Layer (RCL)**
5. **Signature Layer (RSIG)**
6. **Audit Persistence Layer (APL)**

5. Threat Model

5.1 Attacker Capabilities

An attacker MAY attempt:

- replay attacks,
- forgery of CBGMODD role participation,
- manipulation of FAVORS semantic payload,
- inflation of BERA,
- spoofing of resonance engine identity,
- bribery or capture of governance-role signers,
- semantic collision (two distinct intents producing same encoding),
- coerced compliance attacks (forcing a validator to approve false RATE).

5.2 Threat Boundaries

This standard assumes:

- at least one honest verifier exists,
- canonical serialization is correctly implemented,
- cryptographic signature algorithms used are not broken,
- the audit persistence layer is append-only or tamper-evident.

5.3 Security Objectives

A compliant ECS implementation MUST provide:

- **Integrity:** payload modifications invalidate RSIG
- **Replay Resistance:** timestamp and nonce requirements
- **Non-repudiation:** signers cannot deny contribution
- **Auditability:** provenance vectors are preserved
- **Semantic Separation:** two distinct payloads MUST NOT collapse into the same canonical hash unless intentionally identical.

---

6. Canonical RATE Equation

6.1 Normative Definition

RATE SHALL be computed as:

RATE=(CBGMODD×FAVORS)+BERARATE = (CBGMODD \times FAVORS) + BERARATE=(CBGMODD×FAVORS)+BERA

Where:

- CBGMODD is a weighted matrix of governance role participants.
- FAVORS is a standardized semantic vector.
- BERA is a bounded adjustment factor.

6.2 Output Structure

RATE MUST NOT be represented solely as a scalar.

RATE MUST contain:

- **RATE.score** (scalar)
- **RATE.vector** (multi-axis breakdown)
- **RATE.provenance** (inputs hash references)
- **RATE.audit_flags** (policy compliance)
- **RATE.confidence** (verification confidence)

---

7. Canonical Encoding (CEL)

7.1 Serialization Requirements

All ECS payloads MUST be serialized deterministically.

Canonical encoding MUST:

- preserve field ordering,
- enforce numeric precision rules,
- normalize Unicode strings,
- prohibit ambiguous whitespace,
- require explicit null representation.

JSON-C is RECOMMENDED:

- JSON with canonical ordering and strict normalization.

CBOR is OPTIONAL.

7.2 Canonical Payload Hash

A compliant engine MUST compute:

H=HASH(CANONICAL_ENCODE(payload))H = HASH(CANONICAL_ENCODE(payload))H=HASH(CANONICAL_ENCODE(payload))

HASH MUST be SHA-256, SHA-3, or BLAKE3.

---

8. Resonance Evaluation Layer (REL)

8.1 Resonance Constraints

A compliant resonance engine MUST evaluate:

- ethical compliance constraints (CARE),
- ecological constraints (BERC),
- justice constraints (JERC),
- merit contribution constraints (PERC),
- restitution and repair constraints (Paineologic minimization),
- role coherence constraints (CBGMODD validity).

8.2 Pass/Fail Output

REL MUST output:

- pass/fail boolean
- violation list
- compliance score vector
- confidence rating

REL MUST NOT output pass if a critical violation exists.

---

9. Signature Layer (RSIG)

9.1 RSIG Payload

RSIG MUST include:

- payload_hash
- timestamp (UTC)
- nonce (128-bit minimum)
- signer_identity
- engine_identity
- compliance_flags
- RATE.output_hash

9.2 Signature Algorithm Requirements

A compliant system MUST support Ed25519.

It MAY support:

- ECDSA (P-256)
- RSA-PSS

---

10. Audit Persistence Layer (APL)

10.1 Append-Only Requirements

APL MUST be append-only or tamper-evident.

APL MAY be implemented via:

- blockchain,
- merkle-tree anchored log,
- distributed ledger,
- signed audit file system.

10.2 Audit Queryability

A compliant implementation MUST permit retrieval of:

- canonical payload
- RATE object
- RSIG
- verification results

---

11. Interoperability Requirements

11.1 Compatibility with PlayNAC

A compliant ECS system SHOULD expose RATE as a PlayNAC-readable artifact.

11.2 Compatibility with PERC/BERC/JERC

BERA MUST be traceable to BERC/JERC scoring rules.

---

12. Compliance Requirements

A system is **ECS-Compliant** if and only if it:

- implements deterministic encoding,
- computes RATE per Section 6,
- produces RSIG per Section 9,
- preserves provenance vectors,

- supports audit persistence,
- rejects scalar collapse.

---

13. Test Vectors (Draft)

13.1 Placeholder Note

This draft includes placeholder test vectors.
Formal published test vectors MUST be provided in ECS-TEST-2026-001.

Test Vector A (Minimal Citizen Transaction)

CBGMODD:

- Citizen weight = 1.00 credibility = 0.80

FAVORS:

- contribution_units = 10
- benefit_units = 7
- harm_units = 0

BERA:

- +0.50

Expected:

- RATE.score = (1.00×10×0.80) + 0.50 = 8.50

Test Vector B (Multi-Role Resolution)

CBGMODD:

- Citizen: 1.0 credibility 0.7
- Ombudsman: 1.5 credibility 0.9
- Government: 1.2 credibility 0.8

FAVORS:

- resolution_units = 20

BERA:

- +2.0

Expected:

- RATE.score = ((1.0×0.7) + (1.5×0.9) + (1.2×0.8)) × 20 + 2.0
- RATE.score = (0.7 + 1.35 + 0.96) × 20 + 2
- RATE.score = 3.01 × 20 + 2
- RATE.score = 62.2

---

14. Governance and Versioning

14.1 Version Semantics

ECS versions SHALL follow:

- Major = structural breaking change
- Minor = compatible extension
- Patch = editorial or test vector refinement

14.2 Standards Authority

The ERES Institute SHALL serve as the initial custodial authority for ECS.

Downstream institutions MAY fork ECS only if they preserve:

- provenance compatibility,
- canonical serialization requirements,
- audit traceability.

---

15. Ethical Clause (CARE Constraint)

Any implementation of ECS MUST adhere to CARE constraints.

A system MUST NOT use ECS to:

- facilitate punitive exploitation,
- create coercive financial entrapment,
- manufacture false legitimacy,
- conceal ecological destruction.

---

16. Credits

Primary Author: Joseph A. Sprute
Institution: ERES Institute for New Age Cybernetics
Conceptual Contributors:

- ERES governance framework contributors
- CARE-Based Commonwealth Transition model
- EarnedPath and PlayNAC foundational design lineage

---

17. License

This draft is released under:

ERES AuraPath License v1.0
or optionally dual-licensed under **CARE Commons Attribution License (CCAL v2.0)**
(licensing selection SHALL be specified in the final published ECS release).

---

18. References (Informative)

- RFC 2119: Key words for use in RFCs
- SHA-256 / SHA-3 standards
- Ed25519 signature algorithm
- ERES Institute: EarnedPath / PlayNAC / PERC/BERC/JERC / UBIMIA frameworks

---

Appendix A: Standard Terminology Glossary (Draft)

- **CARE**: Cooperative, Accountable, Restorative, Ecologic
- **Paineologic**: Minimization of pain through systemic design
- **Resonance**: measurable alignment of intent, ethics, and outcome
- **Semantic Integrity**: preservation of meaning across transport and time

---

Appendix B: Minimal Implementation Checklist

- Canonical serializer
- RATE engine
- REL evaluator
- RSIG signer
- audit persistence module
- verifier module
- test vector compliance